

21CSC204J- Design and Analysis of Algorithm

Unit 1

Introduction to Algorithm Design

1.1.Fundamentals of Problem Solving

Problem solving is both an art and science. As there are no guidelines available to solve the problem, the problem solving is called an art as high creativity is needed for solving problems effectively. Thus by art, we mean one has to be creative, novel and adventurous and by science, we mean the problem solving should be based on sound mathematical and logical guidelines.

The problem solving stages are given as follows:

1. Problem Understanding
2. Algorithm Planning
3. Design of algorithms
4. Algorithm Verification and Validation
5. Algorithm analysis
6. Algorithm Implementation
7. Perform post-mortem analysis

Problem Understanding:

The problem solving starts with understanding of the problem and aims at providing a detailed problem statement. There should not be any confusion in the problem statement. As the mistakes in understanding the problem may affect the development of algorithms.

Planning of an algorithm

Once problem statement is produced, the planning of algorithm starts. This phase requires selection of computing model and data structures. A computational model is a mathematical model. Data structure is a domain that deals with data storage along with its relationships. Data structures can have impact on the efficiency of the algorithms. Therefore, algorithms and data structures together often constitute important aspect of problem solving. A popular statement in computer science is “Algorithm + Data structure = Program” as a wrong selection of a data structure often proves fatal in problem solving.

Design of Algorithms

Algorithm design is a way of developing the algorithmic solutions using the appropriate design strategy or paradigm.

Algorithm Verification and validation

Algorithm verification and validation is the process of checking algorithm correctness. An algorithm is expected to give correct output for all valid inputs. This process is called algorithm validation. Once validation is over, program proving or program verification starts. Verification is done by

giving mathematical proofs. Mathematical proofs are rigorous and better than scientific methods. Program correctness itself a major study by itself.

Algorithm Analysis

Once the algorithm is proved correct, then the analysis starts. Analysis is important for two reasons. They are as follows:

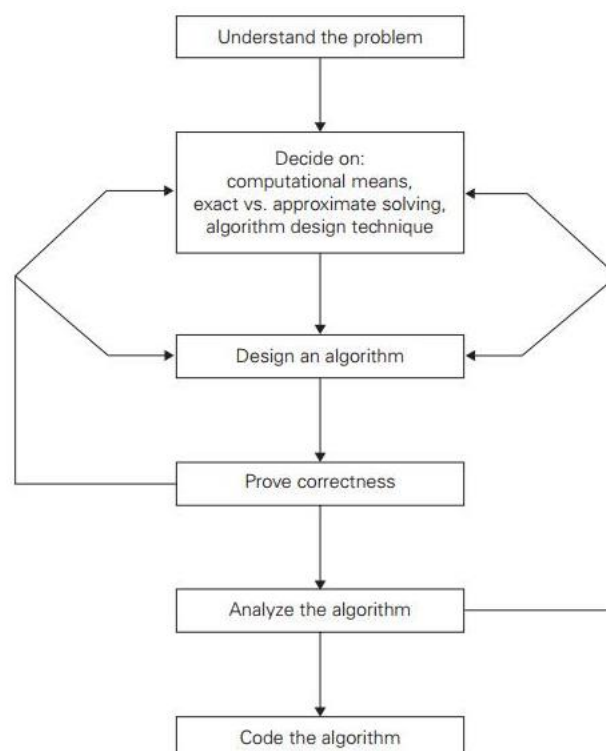
1. To decide efficiency of algorithms
2. To compare algorithms for deciding the effective solutions for a given problem.

Implementation of algorithm as a program and performance analysis

After the algorithms are designed, it is implemented as a program. This stage requires the selection of a programming language. After the program is written, then the program must be tested. Sometimes, the program may not give expected results due to errors. The process of removing the errors is called debugging. Once program is available, they can be verified with a bench mark dataset. It is called experimental algorithmics. This is called performance analysis.

Postmortem analysis

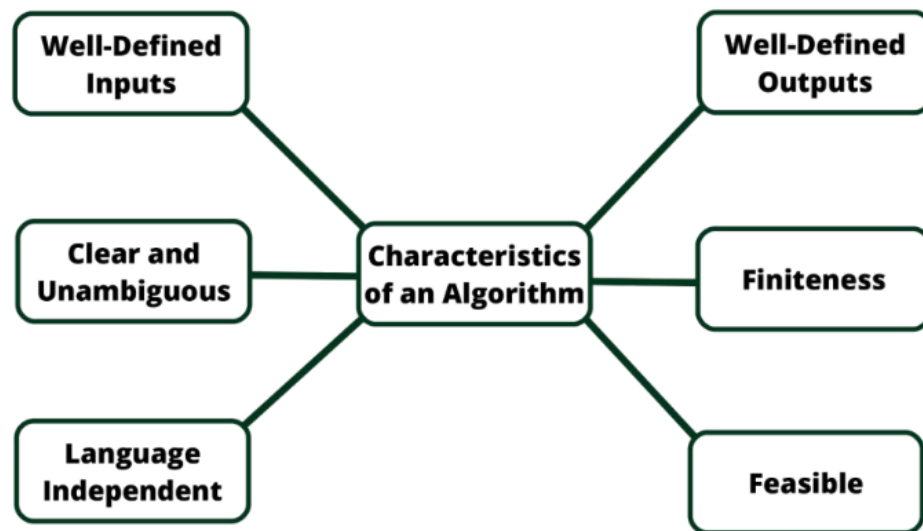
A theoretical best solution for the given problem is called lower bound of the algorithm. The worst case estimate of behavior of the algorithm is called upper bound. The difference between upper and lower bound is called algorithmic gap. Refining the algorithms is to bring these gaps short by reefing planning, design, implementation and analysis. Thus problem solving is not linear but rather this is a cycle.



What is an Algorithm?

A finite set of instructions or logic, written in order, to accomplish a certain predefined task. Algorithm is not the complete code or program. It is just the core logic (solution) of a problem. It can be expressed either as an informal high level description as pseudo code or using a flowchart.

Characteristics of an Algorithm



Well defined Input: An algorithm should have 0 or more well defined inputs.

Well defined Output: An algorithm should have 1 or more well defined outputs

Unambiguous: Algorithm should be clear and unambiguous.

Finiteness: Algorithms must terminate after a finite no. of steps.

Feasibility: Should be feasible with the available resources.

Language Independent: An algorithm should have step-by-step directions which should be independent of any programming code.

What is a Pseudocode?

A Pseudocode is defined as a step-by-step description of an algorithm. Pseudocode does not use any programming language in its representation instead it uses the simple English language text as it is intended for human understanding rather than machine reading.

Pseudocode is the intermediate state between an idea and its implementation(code) in a high-level language



Difference between Algorithm and Pseudocode

Algorithm	Pseudo code
A finite set of instructions or logic, written in order, to accomplish a certain predefined task.	a generic way of describing an algorithm without using any specific programming language-related notations.
It is just the core logic (solution) of a problem	It is an outline of a program, written in a form which can easily be converted into real programming statements.
Easy to understand the logic of a problem	Can be read and understood by programmers who are familiar with different programming languages.
Can be expressed either as an informal high level description as pseudo code or using a flowchart.	It is not written in a specific syntax. It allows to include control structures such as WHILE, IF-THEN-ELSE, REPEAT-UNTIL, FOR, and CASE, which are present in many high level languages.

Why to design an Algorithm?

Designing an algorithm is a fundamental step in problem-solving, especially in computer science, as it provides a structured and efficient way to address complex problems.

1. To Solve Problems Systematically

- An algorithm provides a clear, step-by-step procedure to solve a problem.
- It ensures that the problem is approached logically rather than relying on guesswork or trial-and-error.

2. To Ensure Efficiency

- Well-designed algorithms optimize the use of resources such as time (execution speed) and space (memory usage).
- For large-scale problems, an efficient algorithm can mean the difference between solving a problem in seconds versus hours.

3. To Handle Complexity

- Complex problems, like sorting large datasets, finding shortest paths in graphs, or analyzing big data, require structured approaches that algorithms provide.
- Breaking down a complex problem into smaller, manageable sub-problems is central to algorithm design.

4. To Create Reusable Solutions

- Algorithms can often be generalized and reused for different problems or scenarios.
 - Example: Sorting algorithms (like quick sort, merge sort) can be applied to different datasets.
- This reduces redundancy and development time for similar problems.

5. To Ensure Accuracy and Reliability

- An algorithm, once designed and tested, ensures consistent and accurate results every time it is executed.
- It eliminates human errors in manual computation or ad hoc approaches.

6. To Optimize Decision-Making

- Algorithms help in choosing the best solution among many possibilities.
 - Example: A shortest-path algorithm like Dijkstra's helps find the most efficient route in a network.

7. For Scalability

- As problem size grows, an algorithm ensures the solution remains practical.
- Efficient algorithms handle scalability better, ensuring performance doesn't degrade drastically with larger inputs.

8. To Compare Solutions

- By designing and analyzing algorithms, we can compare multiple approaches to solving the same problem.
- Big-O notation helps assess the efficiency of algorithms in terms of time and space complexity.

9. To Automate Tasks

- Algorithms are key to automation, enabling computers to perform repetitive tasks faster and more reliably than humans.
 - Example: Sorting emails into categories, recommending products, etc.

10. For Innovation and Creativity

- Designing algorithms encourages innovative thinking and problem-solving skills.
- Many modern technologies, such as artificial intelligence and machine learning, are built on creative algorithmic solutions.

Advantages of Algorithm

i) It is easy to understand.

ii) Algorithm is a step-wise representation of a solution to a given problem.

iii) In Algorithm the problem is broken down into smaller pieces or steps hence, it is easier for the programmer to convert it into an actual program.

Disadvantages of Algorithms

i) Writing an algorithm takes a long time so it is time consuming.

ii) Branching and Looping statements are difficult to show in Algorithms.

Real-World Implications of Algorithm Design

- **Search Engines:** Algorithms like PageRank power Google search results.
- **Navigation:** GPS systems use shortest-path algorithms like A* or Dijkstra's.
- **E-Commerce:** Algorithms suggest products or optimize delivery routes.
- **Healthcare:** Algorithms assist in diagnosing diseases and analyzing medical data.

1.2. Correctness of an Algorithm

For any algorithm, we must prove that it always returns the desired output for all legal instances of the problem

How to prove that an algorithm is correct?

Counterexample (indirect proof)

Searching for counterexamples is the best way to disprove the correctness of some things.

- Identify a case for which something is NOT true
- If the proof seems hard or tricky, sometimes a counterexample works
- Sometimes a counterexample is just easy to see, and can shortcut a proof
- If a counterexample is hard to find, a proof might be easier

Proof by Induction

Failure to find a counterexample to a given algorithm does not mean “it is obvious” that the algorithm is correct. Mathematical induction is a very useful method for proving the correctness of recursive algorithms.

1. Prove base case
2. Assume true for arbitrary value n
3. Prove true for case $n + 1$

Loop Invariant

Used to prove the correctness of iterative algorithms.

A loop invariant is a property that holds before and after each iteration of a loop.

Steps:

Initialization: Prove the invariant holds before the first iteration.

Maintenance: Prove that if it holds at the beginning of an iteration, it still holds at the end.

Termination: Show that when the loop exits, the invariant ensures the algorithm produces the correct result.

```
int largest(int arr[], int n) |
{
    int i;
    int max = arr[0];
    for (i = 1; i < n; i++)
        if (arr[i] > max)
            max = arr[i];
    return max;
}
```

Initialization (Base Case)

Before the loop starts ($i = 1$):
 $\text{maxVal} = \text{arr}[0]$. This is correct because, at this point, we have only one element, and the maximum of a single-element array is itself. Thus, the loop invariant holds initially.

Maintenance (Inductive Step)

Assume that before iteration i , maxVal holds the maximum value of $\text{arr}[0..i-1]$.

In the i th iteration, we compare $\text{arr}[i]$ with maxVal :

If $\text{arr}[i] > \text{maxVal}$, we update $\text{maxVal} = \text{arr}[i]$.

Otherwise, maxVal remains unchanged.

After this step, maxVal still contains the maximum of the subarray $\text{arr}[0..i]$.

Thus, the loop invariant is maintained in every iteration.

Termination (Final Step)

The loop runs until $i = n$, meaning we have checked all elements from $\text{arr}[0]$ to $\text{arr}[n-1]$.

At this point, maxVal contains the maximum of the entire array because we updated it whenever we found a larger value.

Thus, the algorithm correctly returns the maximum element.

Other approaches:

Proof by cases/enumeration

Example: Prove $|x| \geq 0$ for all real numbers x .

Case 1: $x > 0$

- If x is positive, then by definition, $|x| = x$.
- Since x is positive, we conclude that $|x| \geq 0$.

Case 2: $x = 0$

- If $x = 0$, then $|x| = 0$.
- Clearly, $|x| \geq 0$.

Case 3: $x < 0$

- If x is negative, then by definition, $|x| = -x$.
- Since x is negative, $-x$ is positive.
- Therefore, $|x| \geq 0$.

Proof by chain of iffs

Example: Prove that a number n is even if and only if n^2 is even.

Assume n is even, meaning $n=2k$ for some integer k .

- Squaring both sides: $n^2=(2k)^2=4k^2=2(2k^2)$

Since $2k^2$ is an integer, n^2 is a multiple of 2, meaning it is **even**.

Thus, **if n is even, then n^2 is even**.

Proof by contradiction

Example : Prove that $\sqrt{2}$ is irrational using proof by contradiction.

Step 1: Assume the Opposite (Contradictory Assumption)

- Assume $\sqrt{2}$ is rational (which means it can be written as a fraction of two integers).
- So, there exist two integers a and b (where $b \neq 0$) such that: $\sqrt{2}=a/b$
- Assume the fraction a/b is in its simplest form (i.e., a and b have no common factors other than 1).

Step 2: Square Both Sides

$$2=a^2/b^2 \quad a^2=2b^2$$

- This means a^2 is an even number (since it is divisible by 2).
- If a^2 is even, then a must also be even (because the square of an odd number is always odd).
- So, we can write a as: $a=2k$ (for some integer k)

Step 3: Substitute $a=2k$ into the Equation

This shows that b^2 is also even, which means b must also be even.

Step 4: Contradiction

- We initially assumed that a and b have no common factors other than 1.
 - But now we have proven that both a and b are even, meaning they both have a common factor of 2, which contradicts our assumption.
 - This contradiction proves that our assumption () was false.
-

Proof by contrapositive

If " P implies Q " and " $\text{Not } Q$ implies $\text{Not } P$ ". For example, the assertion "If it is my car, then it is red" is equivalent to "If that car is not red, then it is not mine". So, to prove "If P , Then Q " by the method of contrapositive means to prove "If $\text{Not } Q$, Then $\text{Not } P$ ".

1.3.Efficiency of an Algorithm

An algorithm is said to be efficient and fast,

- if it takes less time to execute
- consumes less memory space.

The performance of an algorithm is measured on the basis of

- Time Complexity
- Space Complexity

Space Complexity

Problem-solving using computer requires memory to hold temporary data or final result while the program is in execution. The amount of memory required by the algorithm to solve given problem is called space complexity of the algorithm.

The space complexity of an algorithm quantifies the amount of space taken by an algorithm to run as a function of the length of the input. It is the amount of memory needed for the completion of an algorithm.

To estimate the memory requirement we need to focus on two parts:

- (1) A fixed part: It is independent of the input size. It includes memory for instructions (code), constants, variables, etc.
- (2) A variable part: It is dependent on the input size. It includes memory for recursion stack, referenced variables, etc.

Space Complexity (Non Recursive Algorithm)

$$\text{Space complexity-}S(P) = c + S_p \text{ (Non Recursive Algorithm)}$$

©Fixed part component

- Space for Instructions
- Space for variables (independent of input and output)

(S_p) Variable part component

- Space for Reference variables
- Recursion stack space

Eg:1

```
Algo. Swap (a, b)
{
    temp = a;
    a = b;
    b = temp;
}
```

```
Soln
a - 1
b - 1
temp - 1
-----
3
S(P) = 3          O(1)
```

Eg: 2

```
Algo Multiply (A, B, n)
{
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            c[i, j] = 0;
            for (k = 0; k < n; k++)
            {
                c[i, j] = c[i, j] + A[i, k] * B[k, j]
            }
        }
    }
}
```

```
Soln :
A - n2
B - n2
c - n2
-----
i - 1
j - 1
k - 1
n - 1
-----
3n2 + 4
S(P) = 3n2 + 4 ∈ O(n2)
```

// they are matrices

Scalar / simple variables

Space Complexity (Recursive Algorithm)

Space of recursion stack = depth of recursion * (space for 3 components (return Address, Actual Parameter, Local Variable))

Example 1

```
float RSum(float a[], int n)
{
    if n <= 0
        return 0.0
    else
        return (RSum(a, n-1) + a[n])
}
```

Recursion stack space includes :-

Depth of recursion - $(n+1)$

local variable - 0

$(a, n-1)$ } actual parameter - 2

return address - 1

$S(P) = \text{Depth of recursion} \times (\text{local variable} + \text{actual parameter} + \text{return address})$

$S(P) = 3(n+1)$

Eg : 2

```
factorial(n) {
    if (n == 0)
        return 1
    return n * factorial(n-1)
}
```

Depth of Recursion - $n+1$

Local Variable - 0

Actual Parameter - 1

Return Address - 1

$S(P) = \text{Depth of recursion} \times (\text{local variable} + \text{actual parameter} + \text{return address})$

$S(P) = (n+1) * (2)$
 $= 2n + 2 \in \Theta(n)$

1.4. Time Complexity

Time complexity of an algorithm, in general, is simply defined as the time taken by an algorithm to implement each statement in the code. It is not the execution time of an algorithm. This entity can be influenced by various factors like the input size, the methods used and the procedure. An algorithm is said to be the most efficient when the output is produced in the minimal time possible.

Time Complexity $T(P) = c + t_p$

c - Compile Time

- Doesn't depend on instance characteristics
- Compilation is done only once.

t_p - Run Time

- Depends on particular problem instance characteristics
- Concerned and concentrated on Run Time.

Steps to Perform Step Count Analysis

1. **Identify basic operations** (assignments, comparisons, arithmetic operations, function calls).
2. **Count the number of times each operation executes.**
3. **Express the total count as a function of input size nnn.**
4. **Determine the asymptotic complexity** (Big-O notation).

Step Count Analysis

STATEMENT	STEP COUNT
Comments and Declarative	0 Steps
Assignment	1 Step
Conditional	1 Step
Loop Condition (for,while – true-n times and false-1 time)	(n + 1) steps
Body of Loop	n steps

Example 1:

Algo. Swap (a, b)
{
 temp = a;
 a = b;
 b = temp;
}

Algo Swap(a,b)
{
 count = 0; //Assignment statement - 0
 temp = a;
 count++; //Assignment statement - 1
 a = b;
 count++; //Assignment statement - 1
 b = temp;
 count++; //Assignment statement - 1
}

Example 2:

Algo (A, 1, n)
{
 int i;
 for (i = 1 to n)
 A[i] = 0;
}

Algo Assign(A,1,n)
{
 count = 0; //Assignment statement - 0
 int i;
 for(i=1 to n)
 {
 count++; //For statement - n
 A[i] = 0;
 count++; //Assignment statement - n
 }
 count++; //False condition of for - 1
}

In Table Form

Number of Steps per Execution stmt - (s/e)

Number of times each stmt is executed - (Frequency)

Total Contribution of each stmt - [(s/e) * frequency]

stmts	s/e	freq	total steps=s/e*freq
float sum(a[],n)	0	0	0
{			
float s=0.0;	1	1	1
for (int i=1;i<=n;i++)	1	n+1	n+1
{			
s+=a[i];	1	n	n
}			
return s;	1	1	1
TOTAL STEPS			2n+3

O(n)

stmts	s/e	freq	total steps=s/e*freq
matmul(a[],b[])	0	0	
{			
for(int i=1;i<=n;i++)	1	n+1	n+1
for (int j=1;j<=n;j++)	1	n(n+1)	n ² +n
{			
c[i][j]=0;	1	n*n	n ²
for (int k=1;k<=n;k++)	1	n*n(n+1)	n ³ +n ²
{			
c[i][j]=c[i][j]+a[i][k]*b[k][j];	1	n*n*n	n ³
}			
}			
}			
TOTAL STEPS			2n ³ +3n ² +2n+1

O(n³)

1.5.Types of Analysis

- To analyze the given algorithm, it is essential to know with which inputs the algorithm takes less time (performing well) and with which inputs the algorithm takes a long time.
- There are three types of analysis:

Worst case:

- In the worst-case analysis, the upper bound on the running time of an algorithm is calculated.
- The cases that causes a maximum number of operations to be executed must be identified.

Best case:

- In the best-case analysis, the lower bound on the running time of an algorithm is calculated.
- The cases that causes a minimum number of operations to be executed must be identified.

Average case

- Provides a prediction about the running time of the algorithm.
- Run the algorithm many times, using many different inputs that come from some distribution that generates these inputs, compute the total running time (by adding the individual times), and divide by the number of trials.
- **Lower Bound <= Average Time <= Upper Bound**

Example: let $f(n)$ be the function which represents the given algorithm

$$\begin{aligned} f(n) &= n^2 + 500, \text{ for worst case} \\ f(n) &= n + 100n + 500, \text{ for best case} \end{aligned}$$

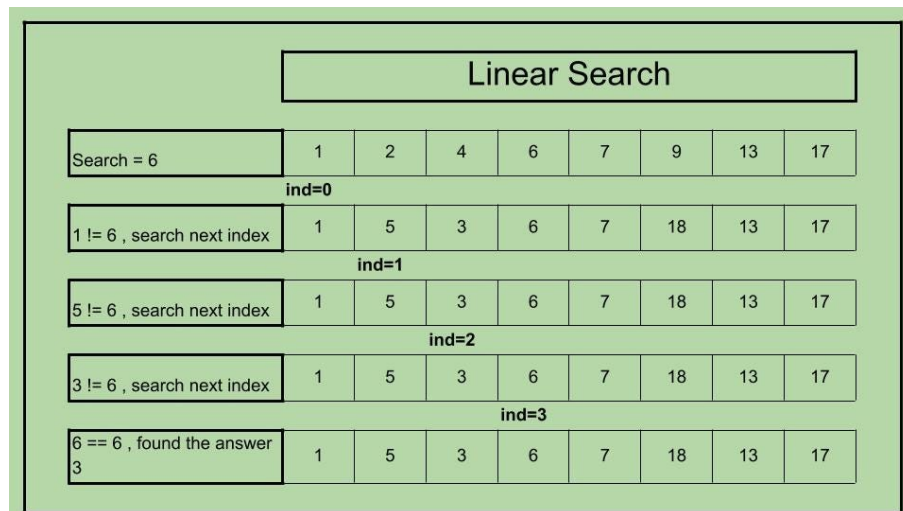
Linear Search

Linear search is also called as sequential search algorithm. It is the simplest searching algorithm. In Linear search, we simply traverse the list completely and match each element of the list with the item whose location is to be found. If the match is found, then the location of the item is returned; otherwise, the algorithm returns NULL.

The steps used in the implementation of Linear Search are listed as follows -

- First, we have to traverse the array elements using a for loop.
- In each iteration of for loop, compare the search element with the current array element, and
 - If the element matches, then return the index of the corresponding array element.
 - If the element does not match, then move to the next element.
- If there is no match or the search element is not present in the given array, return -1.

Working of the Algorithm



Algorithm

```
int Linear_Search(int a[], int n, int key)
{
    int i;
    for (i = 0; i < n; i++)
        if (a[i] == key)
            return i;
    return -1;
}
```

Time Complexity

Best Case Complexity

- The element being searched could be found in the first position.
- In this case, the search ends with a single successful comparison.
- Thus, in the best-case scenario, the linear search algorithm performs $O(1)$ operations.

Worst Case Complexity

- The element being searched may be at the last position in the array or not at all.
- In the first case, the search succeeds in 'n' comparisons.
- In the next case, the search fails after 'n' comparisons.
- Thus, in the worst-case scenario, the linear search algorithm performs $O(n)$ operations.

Average Case Complexity

When the element to be searched is in the middle of the array, the average case of the Linear Search Algorithm is $O(n)$.

Advantages of Linear Search Algorithm:

- Linear search can be used irrespective of whether the array is sorted or not. It can be used on arrays of any data type.
- Does not require any additional memory.
- It is a well-suited algorithm for small datasets.

Disadvantages of Linear Search Algorithm:

- Linear search has a time complexity of $O(N)$, which in turn makes it slow for large datasets.
- Not suitable for large arrays.

When to use Linear Search Algorithm?

- When we are dealing with a small dataset.
- When you are searching for a dataset stored in contiguous memory.

Binary Search

Binary Search Algorithm is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to $O(\log N)$.

Below is the step-by-step algorithm for Binary Search:

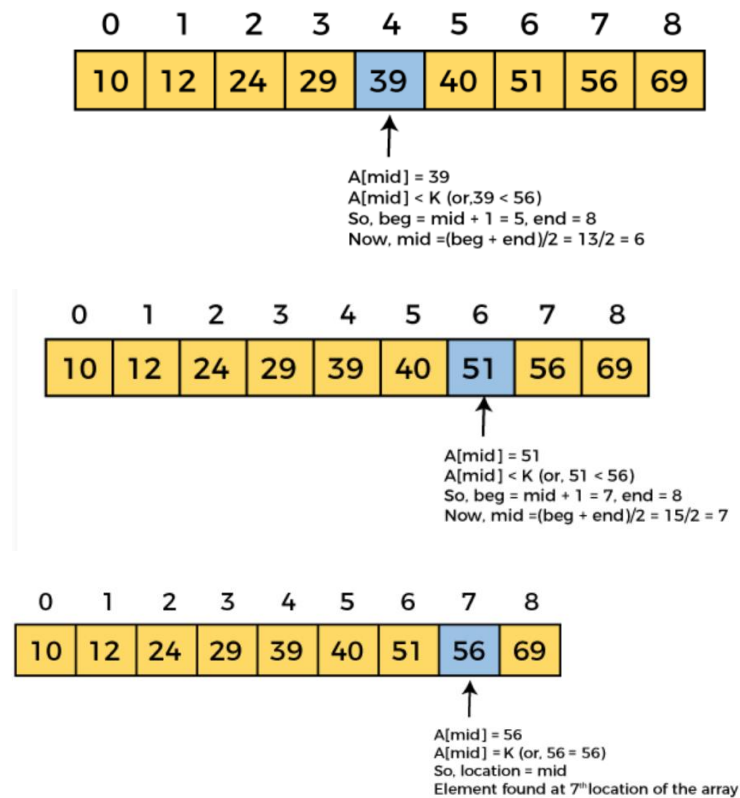
- Divide the search space into two halves by finding the middle index “mid”.
- Compare the middle element of the search space with the key.
- If the key is found at middle element, the process is terminated.
- If the key is not found at middle element, choose which half will be used as the next search space.
 - If the key is smaller than the middle element, then the left side is used for next search.
 - If the key is larger than the middle element, then the right side is used for next search.
- This process is continued until the key is found or the total search space is exhausted.

Working of Binary Search

Let the element to be search (key=56)

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69

$mid = (beg + end)/2$
 $beg = 0, end = 8, mid = (0 + 8)/2 = 4.$
So, 4 is the mid of the array.



The Binary Search Algorithm can be implemented in the following two ways

- Iterative Binary Search Algorithm
- Recursive Binary Search Algorithm (Divide and Conquer – Unit 2)

Iterative Method

```

1  Algorithm BinSearch( $a, n, x$ )
2  // Given an array  $a[1 : n]$  of elements in nondecreasing
3  // order,  $n \geq 0$ , determine whether  $x$  is present, and
4  // if so, return  $j$  such that  $x = a[j]$ ; else return 0.
5  {
6       $low := 1$ ;  $high := n$ ;
7      while ( $low \leq high$ ) do
8      {
9           $mid := \lfloor (low + high)/2 \rfloor$ ;
10         if ( $x < a[mid]$ ) then  $high := mid - 1$ ;
11         else if ( $x > a[mid]$ ) then  $low := mid + 1$ ;
12         else return  $mid$ ;
13     }
14     return 0;
15 }
```

Analysis of Binary Search

Best Case Complexity

In Binary search, best case occurs when the element to search is found in first comparison, i.e., **when the first middle element itself is the element to be searched.** The best-case time complexity of Binary search is $O(1)$.

Worst Case Complexity

- **Case1:** Element is present in the array
- **Case2:** Element is not present in the array.

Each iteration reduces the search space by half:

1. Initially, we have n elements.
2. After 1st iteration, the search space reduces to $n/2$ elements.
3. After 2nd iteration, the search space reduces to $n/4$ elements.
4. After k iterations, the search space reduces to $n / 2^k$ elements.

In the worst case, the search continues until only one element remains:

$$n/2^k=1$$

Taking the logarithm on both sides:

$$\log_2(n/2^k)=\log_2 1$$

Since $\log_2 1=0$, we get:

$$\log_2 n - \log_2 2^k = 0$$

$$\log_2 n - k \log_2 2 = 0$$

Since $\log_2 2=1$, we simplify:

$$k=\log_2 n$$

Thus, the number of iterations (steps) in the worst case is **$O(\log_2 n)$.**

Average Case Complexity

The average case time complexity of Binary search is **$O(\log n)$.**

Advantages of Binary Search:

- **Efficient:** Binary search has a time complexity of **$O(\log n)$** , which makes it very efficient for searching large sorted arrays.
- **Simple to implement:** Binary search is relatively easy to implement and understand.
- **Versatile:** Binary search can be used in a wide variety of applications.

- **Reliable:** Binary search is a reliable algorithm that will always find the target element if it exists in the array.

Disadvantages of Binary Search:

- **Requires a sorted array:** Binary search only works on sorted arrays. If the array is not sorted, it must be sorted before binary search can be used.
- **Not suitable for unsorted data:** Binary search is not suitable for searching unsorted data, as it will not be able to find the target element efficiently.
- **May not be the best choice for large arrays:** For very large arrays (e.g., billions of elements), other search algorithms such as interpolation search or hash tables may be more efficient.

Applications of Binary Search:

- **Searching in sorted arrays:** Binary search is used to efficiently find an element in a sorted array.
- **Database queries:** Binary search can be used to quickly locate records in a database table that is sorted by a specific key.
- **Finding the closest match:** Binary search can be used to find the closest value to a target value in a sorted list.
- **Interpolation search:** Binary search can be used as a starting point for interpolation search, which is an even faster search algorithm.

1.6. Asymptotic Notation

Asymptotic Notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as the input size grows.

These notations provide a concise way to express the behavior of an algorithm's time or space complexity as the input size approaches infinity.

Asymptotic analysis allows for the comparison of algorithms' space and time complexities by examining their performance characteristics as the input size varies.

By using asymptotic notations, such as Big O, Big Omega, and Big Theta, we can categorize algorithms based on their worst-case, best-case, or average-case time or space complexities, providing valuable insights into their efficiency.

From all three notations: worst case, best case, and average case), we can understand that, in every case for a given function $f(n)$ we are trying to find another function $g(n)$ which approximates $f(n)$ at higher values of n . That means $g(n)$ is also a curve which approximates $f(n)$ at higher values of n .

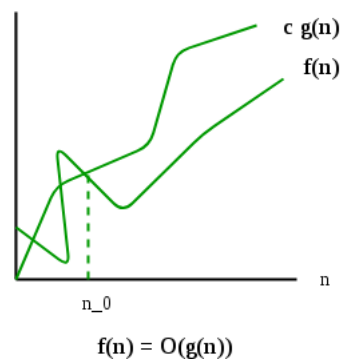
Let us assume that the given algorithm is represented in the form of function $f(n)$.

Big-O Notation [Upper Bounding Function] – Worst Case

- It is the most widely used notation for Asymptotic analysis.

- It specifies the upper bound of a function.
- The maximum time required by an algorithm or the worst-case time complexity.
- It returns the highest possible output value(big-O) for a given input.
- Big-O(Worst Case) It is defined as the condition that allows an algorithm to complete statement execution in the longest amount of time possible.

If $f(n)$ describes the running time of an algorithm, $f(n)$ is $O(g(n))$ if there exist a positive constant C and n_0 such that, $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$



Example:

Example-1 Find upper bound for $f(n) = 3n + 8$

Solution: $3n + 8 \leq 4n$, for all $n \geq 8$
 $\therefore 3n + 8 = O(n)$ with $c = 4$ and $n_0 = 8$

Example-2 Find upper bound for $f(n) = n^2 + 1$

Solution: $n^2 + 1 \leq 2n^2$, for all $n \geq 1$
 $\therefore n^2 + 1 = O(n^2)$ with $c = 2$ and $n_0 = 1$

Example-3 Find upper bound for $f(n) = n^4 + 100n^2 + 50$

Solution: $n^4 + 100n^2 + 50 \leq 2n^4$, for all $n \geq 11$
 $\therefore n^4 + 100n^2 + 50 = O(n^4)$ with $c = 2$ and $n_0 = 11$

Example-4 Find upper bound for $f(n) = 2n^3 - 2n^2$

Solution: $2n^3 - 2n^2 \leq 2n^3$, for all $n > 1$
 $\therefore 2n^3 - 2n^2 = O(n^3)$ with $c = 2$ and $n_0 = 1$

Example-5 Find upper bound for $f(n) = n$

Solution: $n \leq n$, for all $n \geq 1$
 $\therefore n = O(n)$ with $c = 1$ and $n_0 = 1$

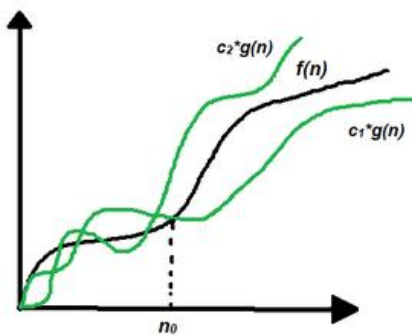
Example-6 Find upper bound for $f(n) = 410$

Solution: $410 \leq 410$, for all $n > 1$
 $\therefore 410 = O(1)$ with $c = 1$ and $n_0 = 1$

Theta notation

Theta notation encloses the function from upper bound and lower bound. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm. Theta (Average Case) the running times for each possible input combination are added and the average is taken as the average case.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be $\Theta(g)$, if there are constants $c_1, c_2 > 0$ and a natural number n_0 such that $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all $n \geq n_0$.



Example 1 Find Θ bound for $f(n) = \frac{n^2}{2} - \frac{n}{2}$

Solution: $\frac{n^2}{5} \leq \frac{n^2}{2} - \frac{n}{2} \leq n^2$ for all, $n \geq 2$
 $\therefore \frac{n^2}{2} - \frac{n}{2} = \Theta(n^2)$ with $c_1 = 1/5, c_2 = 1$ and $n_0 = 2$

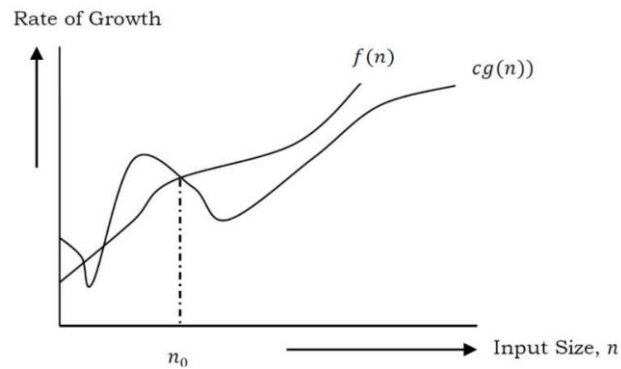
Example 2 Prove $n \neq \Theta(n^2)$

Solution: $c_1 n^2 \leq n \leq c_2 n^2 \Rightarrow$ only holds for: $n \leq 1/c_1$
 $\therefore n \neq \Theta(n^2)$

Omega- Notation [Lower Bounding Function] – Best Case

- It specifies the lower bound of a function.
- The minimum time required by an algorithm or the best-case time complexity.
- It returns the lowest possible output value (big-Omega) for a given input.
- Big-Omega (Best Case) It is defined as the condition that allows an algorithm to complete statement execution in the shortest amount of time possible.

Given two functions $g(n)$ and $f(n)$, we say that $f(n) = \Omega(g(n))$, if there exists constants $c > 0$ and $n_0 \geq 0$ such that $f(n) \geq c * g(n)$ for all $n \geq n_0$.



Similar to the O discussion, this notation gives the tighter lower bound of the given algorithm and we represent it as $f(n) = \Omega(g(n))$. That means, at larger values of n , the tighter lower bound of $f(n)$ is $g(n)$. For example, if $f(n) = 100n^2 + 10n + 50$, $g(n)$ is $\Omega(n^2)$.

Ω Examples

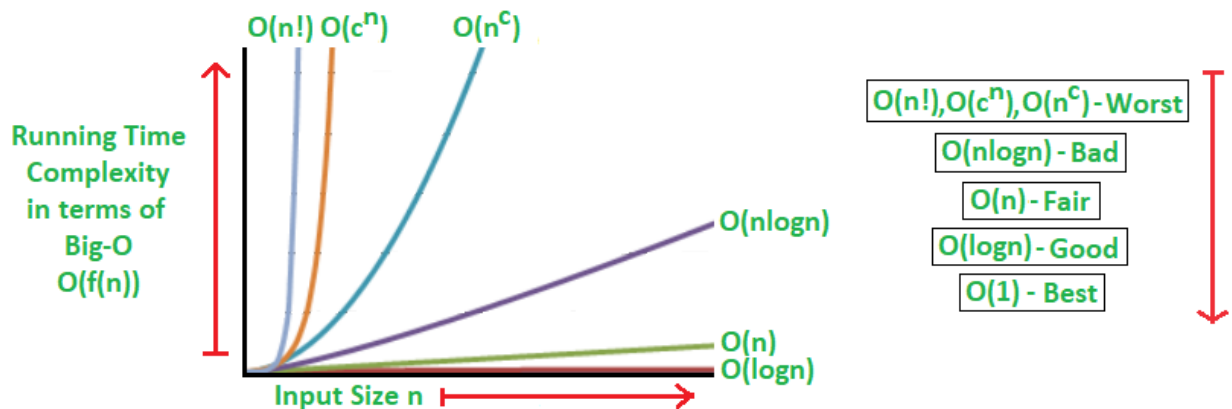
Example-1 Find lower bound for $f(n) = 5n^2$.

Solution: $\exists c, n_0$ Such that: $0 \leq cn^2 \leq 5n^2 \Rightarrow cn^2 \leq 5n^2 \Rightarrow c = 5$ and $n_0 = 1$
 $\therefore 5n^2 = \Omega(n^2)$ with $c = 5$ and $n_0 = 1$

Example-2 Prove $f(n) = 100n + 5 \neq \Omega(n^2)$.

Solution: $\exists c, n_0$ Such that: $0 \leq cn^2 \leq 100n + 5$
 $100n + 5 \leq 100n + 5n(\forall n \geq 1) = 105n$
 $cn^2 \leq 105n \Rightarrow n(cn - 105) \leq 0$
 Since n is positive $\Rightarrow cn - 105 \leq 0 \Rightarrow n \leq 105/c$
 $\Rightarrow$ Contradiction: n cannot be smaller than a constant

Order of Growth



Problem 1:

Prove that $3n+2 \in O(n)$
 $t(n) = 3n+2$; $g(n) = n$.
 To prove that $3n+2 \in O(n)$, the following condition is to be satisfied.
 $t(n) \leq c \cdot g(n)$, $n \geq n_0$
 i.e., $3n+2 \leq c \cdot n$, $n \geq n_0$
 Find constant c, n_0 [non negative integer]
Assume $c=1, n_0=1$, then $n=1, 2, 3, \dots$
 $n=1 \Rightarrow 3(1)+2 \leq 1(1) \quad 5 \leq 1 \quad \times \text{ False}$
Assign $c=2, n_0=1$, then $n=1, 2, 3, \dots$
 $n=1 \Rightarrow 3(1)+2 \leq 2(1) \quad 5 \leq 2 \quad \times \text{ False}$
Assign $c=3, n_0=1$, then $n=1, 2, 3, \dots$
 $n=1 \Rightarrow 3(1)+2 \leq 3(1) \quad 5 \leq 3 \quad \times \text{ False}$
Assign $c=4, n_0=1$, then $n=1, 2, 3, 4, \dots$
 $n=1 \Rightarrow 3(1)+2 \leq 4(1) \quad 5 \leq 4 \quad \times \text{ False}$
 $n=2 \Rightarrow 3(2)+2 \leq 4(2) \quad 8 \leq 8 \quad \checkmark$
 $n=3 \Rightarrow 3(3)+2 \leq 4(3) \quad 11 \leq 12 \quad \checkmark$
 $n=4 \Rightarrow 3(4)+2 \leq 4(4) \quad 14 \leq 16 \quad \checkmark$
 When $c=4$ and $n_0=2$
 $3n+2 \leq 4(n)$ for $n \geq 2$
 Hence the condition is satisfied,
 $3n+2 \in O(n)$ is proved.

Problem 2:

Find upper bound of running time of a cubic function $f(n) = 2n^3 + 4n + 5$.

To find upper bound of $f(n)$, we have to find c and n_0 such that $f(n) \leq c \times g(n)$ for all $n \geq n_0$

$$f(n) \leq c \cdot g(n)$$

$$2n^3 + 4n + 5 \leq c \times g(n)$$

$$2n^3 + 4n + 5 \leq 2n^3 + 4n^3 + 5n^3, \text{ for all } n \geq 1$$

$$2n^3 + 4n + 5 \leq 11n^3$$

So, $c = 11$, $g(n) = n^3$ and $n_0 = 1$

Tabular approach

$$2n^3 + 4n + 5 \leq c \times g(n)$$

$$2n^3 + 4n + 5 \leq 11n^3$$

n	$f(n) = 2n^3 + 4n + 5$	$c \cdot g(n) = 11n^3$
1	11	11
2	29	88
3	71	297
4	149	660

Now, manually find out the proper n_0 , such that $f(n) \leq c \times g(n)$

From Table, for $n \geq 3$, $f(n) \leq c \times g(n)$ holds true. So, $c = 3$, $g(n) = n^3$ and $n_0 = 3$. There can be such multiple pair of (c, n_0)

$f(n) = O(g(n)) = O(n^3)$ for $c = 11$, $n_0 = 1$

$f(n) = O(g(n)) = O(n^3)$ for $c = 3$, $n_0 = 3$ and so on.

Problem 3:

Find lower bound of running time of quadratic function $f(n) = 3n^2 + 2n + 4$.

To find lower bound of $f(n)$, we have to find c and n_0 such that $f(n) \geq c \cdot g(n)$

$$f(n) \geq c \times g(n)$$

$$3n^2 + 2n + 4 \geq c \times g(n)$$

$$3n^2 + 2n + 4 \geq 3n^2 \rightarrow \text{true, for all } n \geq 1$$

Above both inequalities are true and there exists such infinite inequalities.

$$f(n) = \Omega(g(n)) = \Omega(n^2) \text{ for } c = 3, n_0 = 1$$

$$f(n) = \Omega(g(n)) = \Omega(n^2) \text{ for } c = 1, n_0 = 1$$

and so on.

Problem 4:

Find tight bound of running time of quadratic function $f(n) = 6 \cdot 2^n + n^2$

$$f(n) = 6 \cdot 2^n + n^2$$

To prove above statement, we have to find c_1 , c_2 and n_0

such that, $0 \leq c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$ for all $n \geq n_0$

$$c_1 \times g(n) \leq 6 \cdot 2^n + n^2 \leq c_2 \times g(n)$$

$$6 \cdot 2^n \leq 6 \cdot 2^n + n^2 \leq 7 \cdot 2^n, \text{ for all } n \geq 1$$

$$\text{So, } f(n) = \Theta(g(n)) = \Theta(2^n) \text{ for } c_1 = 6, c_2 = 7, n_0 = 1$$

1.7. Insertion Sort

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list.

It is a Stable sorting

Why the name Insertion Sort?

- At each step, an element is **inserted** into its correct position among the previously sorted elements.
- The process resembles how people sort playing cards in their hands —by picking one card at a time and inserting it into the correct spot.



Working of the Algorithm

Sort the given array [10, 4, 8, 6, 1]. Divide the array into two subsets of sorted and unsorted lists to demonstrate subsets. Iteration starting with the first index [0] as below.

0

10	4	8	6	1
----	---	---	---	---

There are no numbers to compare index $[0] = 10$, so setting the first index $[0] = 10$ as the sorted subset and other numbers as Unsorted subset.

10	4	8	6	1
----	---	---	---	---

1.

10	4	8	6	1
----	---	---	---	---

The first unsorted subset element index $[1] = 4$ will be compared with the sorted subset element 10. As, $10 > 4$, the element 10 will be moved to right and 4 will be moved into the sorted subset

10	4	8	6	1
----	---	---	---	---

4	10
---	----

→ Sorted array

8	6	1
---	---	---

→ Unsorted array

2.

4	10	8	6	1
---	----	---	---	---

Now, we have 2 numbers in a sorted array and 3 numbers in an unsorted array. The first element in the unsorted array index $[2] = 8$ will be compared to its left side elements $[4, 10]$ to find its rightful sorted position.

As, $10 > 8$, element 10 will be moved right and 8 will be inserted in its index position. Now, $8 > 4$. So, it will remain in its position as the array is sorted.

4	10	8	6	1
---	----	---	---	---

4	8	10
---	---	----

→ Sorted array

6	1
---	---

→ Unsorted array

3.

4	8	10	6	1
---	---	----	---	---

If you see above, the sorted subset is increasing, and the unsorted subset is shrinking. Continue the iteration considering the unsorted subset index $[3] = 6$ as the number to be compared to its left side sorted arrays.

As, $10 > 6$, move 10 to the right side and insert 6 in the sorted list.

The next element in the sorted list is 8, as $8 > 6$, 8 will be moved to the right, and 6 will be inserted in the place of 8. The next element is 4, which is less than 6, so it's safe to say that number 6 is in its rightful sorted position.

4	8	10	6	1
---	---	----	---	---

4	8	6	10	1
---	---	---	----	---

4	6	8	10
---	---	---	----

→ Sorted array

1

→ Unsorted array

4.

4	6	8	10	1
---	---	---	----	---

The unsorted subset has only one number. To find its rightful position, the index $[4] = 1$ will be compared against unsorted subset numbers.

The leftmost number to index $[4]$ is 10 which is greater than 1, so 10 is moved right and 1 is inserted in the place of 10.

The next element is 8, which is also greater than 1. It is moved to the right, and 1 will be inserted in index $[2]$.

The next element to the left of number 1 is 6, as $6 > 1$, the greater number is moved right and $[1]$ will take a position in index $[1]$.

The next element to the left of number 1 is 4, as $4 > 1$, it is moved to the right in index $[1]$ and 1 will be inserted in index $[0]$.

4	6	8	10	1
---	---	---	----	---

4	6	8	1	10
---	---	---	---	----

4	6	1	8	10
---	---	---	---	----

4	1	6	8	10
---	---	---	---	----

1	4	6	8	10
---	---	---	---	----

→ Final Sorted array

Algorithm

```
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

1.8. Analysis of the Algorithm

Insertion Sort	Best Case(Already Sorted)			Worst Case (Descending Order)		
	s/e	Frequency	Total Steps	s/e	Frequency	Total Steps
void insertionSort(int arr[], int n) {	-		-	-		-
for (int i = 1; i < n; ++i)	1	n [n-1(True) +1(False)]	n	1	n [n-1(True) +1(False)]	n
{						
int key = arr[i];	1	n-1	n-1	1	n-1	n-1
int j = i - 1;	1	n-1	n-1	1	n-1	
while (j >= 0 && arr[j] > key) {	1	Will not be true -	-	1	True $1+2+3+\dots+(n-1)=n(n-1)/2 - 1$	n^2
arr[j + 1] = arr[j];	1	-	-	1	$n(n-1)/2 - 1$	n^2
j = j - 1; }	1	-	-	1	$n(n-1)/2 - 1$	n^2
arr[j + 1] = key;}	1	-		1	n-1	n-1
}						
Total			2(n-1)			$O(n^2)$

Time Complexity of Insertion Sort

Best Case : $O(n)$ Worst Case : $O(n^2)$ Average Case : $O(n^2)$
--

Advantages of Insertion Sort:

- Simple and easy to implement.

- Stable sorting algorithm. (A stable sorting algorithm preserves the relative order of records with equal keys, or values. This means that if two records have the same key, the order of those records remains the same after sorting.)
- Efficient for small lists and nearly sorted lists.
- Space-efficient as it is an in-place algorithm. (sorts the input data directly within the original data structure, without requiring any additional memory space proportional to the input size)

Disadvantages of Insertion Sort:

Inefficient for large lists.

Not as efficient as other sorting algorithms (e.g., merge sort, quick sort) for most cases.

1.9. Different Design Paradigms / Techniques

Algorithm Design paradigms in computer science and software development refer to overarching approaches or strategies for solving problems and designing algorithms. Each paradigm has distinct characteristics and is suited to specific types of problems.

Divide and Conquer

- **Description:** Breaks a problem into smaller sub-problems, solves each sub-problem independently, and combines their results. It is a top-down approach.
- **Key Idea:** Recursion and splitting the input.
- **Examples:**
 - Merge Sort
 - Quick Sort
 - Binary Search
 - Fast Fourier Transform (FFT)

Dynamic Programming (DP)

- **Description:** Solves problems by breaking them into overlapping sub-problems and solving each sub-problem only once, storing the result for future reference.
 - ❖ In divide and conquer, many times the recursively solved subproblems can result in the same computation being performed multiple times. This problem arises when identical problems occur repeatedly in a recursion.
 - ❖ Dynamic programming is used to avoid this issue by storing the results of subproblems in a table and referring to that table to check if we have already calculated the solution to a subproblem before computing it again
- **Key Idea:** Optimization using memoization (top-down) or tabulation (bottom-up).
- **Examples:**
 - Fibonacci sequence
 - Longest Common Subsequence (LCS)
 - Knapsack problem
 - Matrix Chain Multiplication

Greedy Algorithms

- **Description:**
 - ❖ Greedy method is used to solve the optimization problem.
 - ❖ An optimization problem is one in which we are given a set of input values, which are required either to be maximized or minimized (known as objective), i.e. some constraints or conditions.
 - ❖ Makes a locally optimal choice at each step with the hope that this leads to a globally optimal solution.
 - ❖ Greedy Algorithm always makes the choice (greedy criteria) looks best at the moment, to optimize a given objective.

- ❖ The greedy algorithm doesn't always guarantee the optimal solution however it generally produces a solution that is very close in value to the optimal.
- **Key Idea:** Optimization problems, Simplicity and no backtracking.

- **Examples:**

Dijkstra's algorithm (shortest path)

Prim's algorithm (minimum spanning tree)

Huffman encoding (data compression)

Backtracking

- **Description:** Builds a solution incrementally and abandons a solution ("backtracks") as soon as it determines that it cannot lead to a valid solution.
- **Key Idea:** Depth-first search with pruning.
- **Examples:**
 - N-Queens problem
 - Sudoku solver
 - Subset sum problem

Branch and Bound

- **Description:** Similar to backtracking but uses bounds to eliminate large portions of the search space, improving efficiency. More than 1 solution is possible.
- **Key Idea:** Pruning based on a bounding function.
- **Examples:**
 - Traveling Salesman Problem (TSP)
 - Knapsack problem

Brute Force

- **Description:** Examines all possible solutions to find the correct one.
- **Key Idea:** Exhaustive search.
- **Examples:**
 - Password cracking
 - Finding all permutations of a string

Randomized Algorithms

- **Description:** Uses random numbers to influence decisions during execution.
- **Key Idea:** Randomness improves performance or simplifies logic.
- **Examples:**
 - Randomized Quick Sort
 - Monte Carlo methods
 - Hashing

1.10. Comparison of Greedy, Divide and Conquer, Dynamic Programming

Greedy Algorithm	Divide and conquer	Dynamic Programming
Follows Top-down approach	Follows Top-down approach	Follows bottom-up approach
Used to solve optimization problem	Used to solve decision problem	Used to solve optimization problem
The optimal solution is generated without revisiting previously generated solutions; thus, it avoids the re-computation	Solution of subproblem is computed recursively more than once.	The solution of subproblems is computed once and stored in a table for later use.
It may or may not generate an optimal solution.	It is used to obtain a solution to the given problem, it does not aim for the optimal solution	It always generates optimal solution.
Iterative in nature.	Recursive in nature.	Recursive in nature.
Efficient and fast than divide and conquer.	Less efficient and slower.	More efficient but slower than greedy.
Extra memory is not required.	Some memory is required.	More memory is required to store subproblems for later use.

Comparison of Backtracking, Branch and Bound

Parameters	Backtracking	Branch-N-Bound
Why it is used?	This is used to solve the decision-based problem.	This is used to solve the optimization problem.
Nodes	This is a state space tree where the node explored the depth-first search.	This explored the optimization problem.
Efficient	More efficient.	Less Efficient.
Function	It involves the feasibility function.	It involves the bounding function.
Traverse	It traverses the tree by depth-first search	It traverses the tree by breadth-first search
Solves	Backtracking can solve the game of chess and sudoku.	It doesn't solve any game problem.
Application Used	This application is used to solve the N-queen problem, the Hamilton cycle, and the problem based on graph coloring.	This type of application is used to solve the problem based on the Travelling salesman problem.

Advantages of Greedy Algorithms

Simple and easy to understand: Greedy algorithms are often straightforward to implement and reason about.

Efficient for certain problems: They can provide optimal solutions for specific problems.

Fast execution time: Greedy algorithms generally have lower time complexity compared to other algorithms for certain problems.

Intuitive and easy to explain : The decision-making process in a greedy algorithm is often easy to understand and justify.

Can be used as building blocks for more complex algorithms: Greedy algorithms can be combined with other techniques to design more sophisticated algorithms for challenging problems.

Disadvantages of the Greedy Approach

Not always optimal: Greedy algorithms prioritize local optima over global optima, leading to suboptimal solutions in some cases.

Difficult to prove optimality: Proving the optimality of a greedy algorithm can be challenging, requiring careful analysis.

Sensitive to input order: The order of input data can affect the solution generated by a greedy algorithm.

Limited applicability: Greedy algorithms are not suitable for all problems

Advantages of Divide and Conquer:

- **Efficiency:** Divide and conquer algorithms typically have a time complexity of $O(n \log n)$, which is more efficient than many other algorithms for large datasets.
- **Simplicity:** Divide and conquer algorithms are often easy to understand and implement.
- **Parallelizability:** Divide and conquer algorithms can be easily parallelized, as each subproblem can be solved independently.
- **Cache-friendliness:** Divide and conquer algorithms tend to have good cache performance, as they access data in a predictable pattern.

Disadvantages of Divide and Conquer:

- **Recursion overhead:** Divide and conquer algorithms use recursion, which can lead to significant overhead in terms of stack space and function calls.
- **Not suitable for all problems:** Divide and conquer algorithms are not suitable for all types of problems. They are most effective for problems that can be recursively divided into smaller subproblems.
- **Limited memory efficiency:** Divide and conquer algorithms can require a significant amount of memory, as they create multiple copies of the input data.
- **Difficult to analyze:** The time and space complexity of divide and conquer algorithms can be difficult to analyze, especially for complex problems.

Recursion is a method to solve a problem where the solution depends on solutions to smaller subproblems of the same problem. Recursive functions (function calling itself) are used to solve problems based on Recursion. The main challenge with recursion is to find the time complexity of the Recursive function.

Advantages of Dynamic Programming

Efficiency: Reduces the time complexity by avoiding redundant calculations through the reuse of already computed solutions.

Optimal Solutions: Ensures that the solutions to problems are optimal by solving and combining the results of subproblems.

Handles Complex Problems: Can solve problems that are difficult or impossible to solve using other techniques, especially those with overlapping subproblems.

Improves Recursive Algorithms: Converts exponential-time recursive algorithms into polynomial-time solutions.

Disadvantages of Dynamic Programming

High Space Complexity: Can require significant memory to store solutions for all subproblems, Complexity in Formulation: Identifying subproblems, defining recurrence relations, and setting up base cases can be challenging and may require deep insight into the problem.

Difficult to Debug: Debugging dynamic programming solutions can be harder due to the complexity of the state transitions and table updates.

May Lead to Suboptimal Performance: If not carefully implemented (e.g., unnecessary recalculations or improper state representation), DP can still result in inefficient solutions.

Advantages of Backtracking Algorithms

- **Simple and Intuitive:** Easy to understand and implement for a wide range of problems.
- **Flexibility:** Can be applied to many types of problems, especially combinatorial and constraint satisfaction problems.
- **Exhaustive Search:** Guarantees finding a solution if one exists by exploring all possible options.
- **Effective Pruning:** Can reduce the search space significantly by eliminating paths that cannot lead to a solution.
- **Handles Complex Problems:** Suitable for problems with complex constraints

Disadvantages of Backtracking Algorithms

- **High Time Complexity:** Can be very slow due to the need to explore all possible solutions, leading to exponential time complexity in many cases.
- **Memory Intensive:** Uses a lot of memory, especially for deep recursion and large decision trees.
- **Inefficient for Large Problems:** Not practical for large-scale problems due to the combinatorial explosion of possibilities.
- **Non-Optimal Solutions:** While it finds a solution, it may not be the most efficient or optimal one without further optimization.
- **Requires Careful Pruning:** Effective use of backtracking depends on good pruning techniques, which can be complex to implement correctly.

Advantages of Branch and Bound

- **Time Complexity:** The Branch and Bound algorithm does not explore all nodes in the tree. Thereby, the time complexity is significantly lesser than most other algorithms.

- **Optimal Solution:** If the branching is done reasonably, the algorithm can find the optimal solution in a reasonable period.
- **Clear Pattern:** The Branch and Bound algorithm does not repeat the nodes to explore the tree for the candidate solutions; instead, it follows a minimal path to derive the optimal solution.

Disadvantages of Branch and Bound

- **Time-consuming:** Based on the size of the problem, the number of nodes that are computed might be too large in a worst-case scenario, making it a time-consuming process.
- **Parallelization:** The branching out of possible solutions provides scope for speculative parallelism. However, when alternative actions are considered for the said action, the branch and bound calculator finds difficulty.

1.11. Mathematical Induction

Mathematical Induction is a technique of **proving a statement, theorem or formula which is thought to be true, for each and every natural number n**. By generalizing this in form of a principle which we would use to prove any mathematical statement is 'Principle of Mathematical Induction'.

Mathematical Induction Steps

Base Step: Prove $P(k)$ is true for $k = 1$

Assumption Step: Let $P(k)$ is true for all k in N and $k > 1$

Induction Step: Prove $P(k+1)$ is true using basic mathematical properties.

Example 1:

For any positive integer number n, prove that $n^3 + 2n$ is always divisible by 3

Solution:

Let $P(n)$: $n^3 + 2n$ is divisible by 3 be the given statement.

Step 1: Basic Step

Firstly we prove that $P(1)$ is true. Let $n = 1$ in $n^3 + 2n$
 $= 1^3 + 2(1)$
 $= 3$

As 3 is divisible by 3. Hence, $P(1)$ is true.

Step 2: Assumption Step

Let us assume that $P(k)$ is true

Then, $k^3 + 2k$ is divisible by 3

Thus, we can write it as $k^3 + 2k = 3n$, (where n is any positive integer)...(i)

Step 3: Induction Step

Now we have to prove that algebraic expression $(k + 1)^3 + 2(k + 1)$ is divisible by 3
 $= (k + 1)^3 + 2(k + 1)$
 $= k^3 + 3k^2 + 5k + 3$
 $= (k^3 + 2k) + (3k^2 + 3k + 3)$
 $= 3n + 3(k^2 + k + 1)$ from eq(i)
 $= 3(n + k^2 + k + 1)$

As it is a multiple of 3 we can say that it is divisible by 3.

Thus, $P(k+1)$ is true i.e. $(k+1)^3 + 2(k+1)$ is divisible by 3.

Now by the Principle of Mathematical Induction, we can say that, $P(n)$: $n^3 + 2n$ is divisible by 3 is true.

Example 2: Prove that the formula for the sum of n natural numbers holds true for all natural numbers, that is, $1 + 2 + 3 + 4 + 5 + \dots + n = n(n+1)/2$ using the principle of mathematical induction.

Solution:

Suppose $P(n)$: $1 + 2 + 3 + 4 + 5 + \dots + n = n(n+1)/2$

Here we use the concept of mathematical induction and prove this across the following three steps.

Base Step: To prove $P(1)$ is true.

For $n = 1$, LHS = 1

RHS = $1(1+1)/2 = 2/2 = 1$

Hence LHS = RHS $\Rightarrow P(1)$ is true.

Assumption Step: Assume that $P(n)$ holds for $n = k$, i.e., $P(k)$ is true
 $\Rightarrow 1 + 2 + 3 + 4 + 5 + \dots + k = k(k+1)/2$ --- (1)

Induction Step: Now we will prove that $P(k+1)$ is true.

To prove: $1 + 2 + 3 + 4 + \dots + (k+1) = (k+1)(k+2)/2$

Consider LHS = $1 + 2 + 3 + 4 + \dots + (k+1)$

= $1 + 2 + 3 + 4 + \dots + k + (k+1)$

= $(1 + 2 + 3 + 4 + \dots + k) + k+1$

= $k(k+1)/2 + k+1$ [Using (1)]

= $[k(k+1) + 2(k+1)]/2$

= $(k+1)(k+2)/2$

= RHS

$\Rightarrow P(n)$ is true for $n = k+1$

Hence, by the principle of mathematical induction, $P(n)$ is true for all natural numbers n .

Example 3: Prove that $2n > n$ for all positive integers n .

Solution: We will prove the result using the principle of mathematical induction.

Assume $P(n)$: $2n > n$

Base Step: To prove $P(1)$ is true.

For $n = 1$, we have $2 \times 1 = 2 > 1$

$\Rightarrow P(1)$ is true.

Assumption Step: Assume that $P(n)$ holds for $n = k$, i.e., $P(k)$ is true
 $\Rightarrow 2k > k$ --- (1)

Induction Step: Now we will prove that $P(k+1)$ is true.

To prove: $2k+1 > k+1$

Consider $2k+1$

$$= 2 \cdot 2k$$

$$> 2k \text{ [Using (1)]}$$

$$= k + k$$

$$> k + 1 \text{ [Because any natural number other than 1 is greater than 1.]}$$

$$\Rightarrow P(n) \text{ is true for } n = k+1$$

Hence, by the principle of mathematical induction, $P(n)$ is true for all natural numbers n .

Example 4: Prove the following formula using the Principle of Mathematical Induction. $12 + 32 + 52 + \dots + (2n - 1)^2 = \frac{n(2n-1)(2n+1)}{3}$

Solution: Assume $P(n): 12 + 32 + 52 + \dots + (2n - 1)^2 = \frac{n(2n-1)(2n+1)}{3}$

Base Step: To prove $P(1)$ is true.

$$\text{For } n = 1, \text{ LHS} = 12 = 1$$

$$\text{RHS} = \frac{1(2 \times 1 - 1)(2 \times 1 + 1)}{3} = \frac{1(2-1)(2+1)}{3} = \frac{3}{3} = 1$$

$$\text{Hence LHS} = \text{RHS} \Rightarrow P(1) \text{ is true.}$$

Assumption Step: Assume that $P(n)$ holds for $n = k$, i.e., $P(k)$ is true

$$\Rightarrow 12 + 32 + 52 + \dots + (2k - 1)^2 = \frac{k(2k-1)(2k+1)}{3} \text{ --- (1)}$$

Induction Step:

Now we will prove that $P(k+1)$ is true.

$$\text{To prove: } 12 + 32 + 52 + \dots + (2k - 1)^2 + [2(k+1) - 1]^2 = \frac{(k+1)[2(k+1)-1][2(k+1)+1]}{3}$$

$$\text{Consider LHS} = 12 + 32 + 52 + \dots + (2k - 1)^2 + [2(k+1) - 1]^2$$

$$= [12 + 32 + 52 + \dots + (2k - 1)^2] + [2(k+1) - 1]^2$$

$$= \frac{k(2k-1)(2k+1)}{3} + [2(k+1) - 1]^2 \text{ [Using (1)]}$$

$$= \frac{k(2k-1)(2k+1)}{3} + (2k+1)^2$$

$$= \frac{[k(2k-1)(2k+1) + 3(2k+1)^2]}{3}$$

$$= \frac{(2k+1)[k(2k-1) + 3(2k+1)]}{3}$$

$$= \frac{(2k+1)[2k^2 - k + 6k + 3]}{3}$$

$$= \frac{(2k+1)[2k^2 + 5k + 3]}{3}$$

$$= \frac{(2k+1)[2k^2 + 2k + 3k + 3]}{3}$$

$$= \frac{(2k+1)[2k(k+1) + 3(k+1)]}{3}$$

$$= \frac{(2k+1)(2k+3)(k+1)}{3}$$

$$= \frac{(k+1)[2(k+1)-1][2(k+1)+1]}{3}$$

$$= \text{RHS}$$

$$\Rightarrow P(n) \text{ is true for } n = k+1$$

Hence, by the principle of mathematical induction, $P(n)$ is true for all natural numbers n .

1.12. What is a Recurrence Relation?

Whenever any function makes a recursive call to itself, its time can be computed by a Recurrence Relation. Recurrence Relation is simply a mathematical relation/equation that can give the value of any term in terms of some previous smaller terms. For example,

$$T(n) = T(n-1) + N.$$

It is a recurrence relation because the value of the n th term is given in its previous term i.e. $(n-1)$ th term.

Formulating Recurrence Relation

Formulating a recurrence relation involves expressing a problem in terms of smaller instances of itself. Here are the key steps in the formulation process:

Steps to Formulate a Recurrence Relation:

1. **Identify the problem size (n):** Determine how the input size reduces in recursive calls.
2. **Express the problem in terms of sub problems:** Break the problem into smaller instances and relate them mathematically.
3. **Define base cases:** Establish stopping conditions for recursion (typically small values of n where direct computation is possible).
4. **Write the recurrence relation:** Formulate the equation representing the problem using smaller sub problems.

Example 1:Framing Recurrence relation for Factorial computation Algorithm

The **factorial** of a number n , denoted as $n!$, is the product of all positive integers from 1 to n . It is defined as: $n! = n \times (n-1)!$

For example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Step-by-Step Breakdown for Recurrence Relation:

1. **Identify the Problem Size (n)**
 - The input size is n , the number for which we need to compute the factorial.
2. **Break the Problem into Subproblems**
 - The factorial of n is dependent on the factorial of $(n-1)$:
 - $n! = n \times (n-1)!$
 - The problem reduces to computing $(n-1)!$, making it a **recursive** problem.
3. **Define the Base Case**
 - The factorial of **0 or 1** is defined as **1**:
 - $0! = 1$
 - $1! = 1$
 - **Base case:** $T(1) = O(1)$
4. **Write the Recurrence Relation**

Since computing $n!$ involves a single multiplication with $(n-1)!$, the recurrence relation is:

$$T(n) = T(n-1) + O(1)$$

Final Recurrence Relation for Factorial:

$$T(n)=T(n-1)+O(1)$$

with the base case:

$$T(1)=O(1)$$

Example 2:

Framing Recurrence relation for Binary Search

Binary Search is an efficient algorithm for searching an element in a sorted array. It works by repeatedly dividing the array in half and checking whether the middle element is the target. If not, it eliminates one half of the array and continues searching in the remaining half.

Step-by-Step Breakdown for Recurrence Relation:

1. **Identify the problem size (n)**
 - The input size is n (the number of elements in the sorted array).
2. **Break the problem into sub problems**
 - At each step, Binary Search compares the middle element $A[mid]$ with the target:
 - If $A[mid] == \text{target} \rightarrow$ Found, return index.
 - If $A[mid] > \text{target} \rightarrow$ Search in the **left half**.
 - If $A[mid] < \text{target} \rightarrow$ Search in the **right half**.
 - Since we only search one half of the array in each step, the problem size reduces to $n/2$.
3. **Define the Base Case**
 - When $n=1$ we directly compare the element: $T(1)=O(1)$
4. **Write the Recurrence Relation**
 - The time complexity function $T(n)$ depends on:
 - A single comparison (constant time $O(1)$).
 - Recursively searching in one half of the array.

$$T(n)=T(n/2)+O(1)$$

Final Recurrence Relation for Binary Search:

$$T(n)=T(n/2)+O(1)$$

with the base case:

$$T(1)=O(1)$$

Example 3:

Framing Recurrence relation for Merge Sort

- The input size is n , the number of elements in the array.
- 2. **Break the Problem into Sub problems**
 - The array is **divided into two halves**:
 - The first half has $n/2$ elements.
 - The second half has $n/2$ elements.
 - We recursively sort **both halves**.
- 3. **Work Done at Each Step**
 - **Dividing the array**: Takes **constant time** $O(1)$.
 - **Recursively sorting both halves**: Takes $2T(n/2)$.
 - **Merging the two sorted halves**: Takes $O(n)$ time.
 - Total work done = sorting both halves + merging them.
- 4. **Write the Recurrence Relation**

Since we divide the problem into two subproblems of size $n/2$ and do $O(n)$ work in merging, the recurrence relation is:

$$T(n) = 2T(n/2) + O(n)$$
- 5. **Define the Base Case**
 - When $n=1$, the array is already sorted $\rightarrow$ Takes **constant time**.
 - **Base case**: $T(1) = O(1)$

Final Recurrence Relation for Merge Sort:

$$T(n) = 2T(n/2) + O(n)$$

with the base case:

$$T(1) = O(1)$$

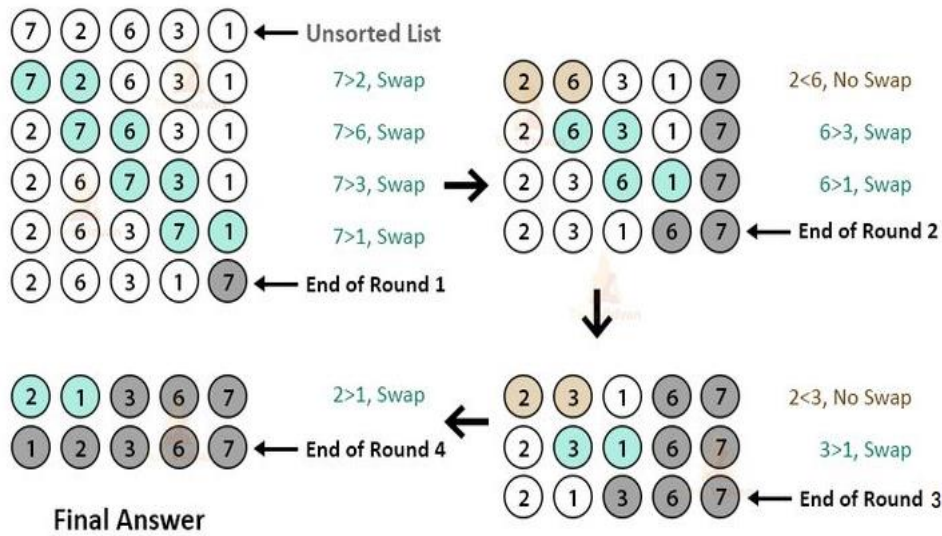
1.13. Bubble Sorting Algorithm and its Analysis

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets. It is a stable sorting.

The **bubble sort** makes multiple passes through a list. It compares adjacent items and exchanges those that are out of order. Each pass through the list places the next largest value in its proper place. In essence, each item “bubbles” up to the location where it belongs.

The working of the Bubble sort is shown below. After the end of Pass 1 / Round 1, the largest element will reach its proper place. After the end of Pass 2 / Round 2, the second largest element will reach its proper place.

Working of Bubble Sort



Algorithm

```
void bubbleSort(arr[], n)
{
    int i,j,temp;
    for (i = 0; i < n-1; i++)
    {
        for (j = 0; j < n-i-1; j++)    // Last i elements are already in place
        {
            if(arr[j] > arr[j+1])
            {
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
            }
        }
    }
}
```

Bubble Sort	Best Case(Already Sorted)			Worst Case (Descending Order)		
	s/e	Frequency	Total Steps	s/e	Frequency	Total Steps
void bubbleSort(arr[], n)	-		-	-		-
{	-					
int i,j,temp;	-					
for (i = 0; i < n-1; i++)	1	n [n-1(True) +1(False)]	n	1	n [n-1(True) +1(False)]	n
{						
for (j = 0; j < n-i-1; j++)	1	n-1	n-1	1	n-1	n-1
{						
if(arr[j] > arr[j+1]) {	1	n-1	n-1	1	n-1	n-1

temp=a[j];	1	Will not be true -	-	1	True $1+2+3+\dots+(n-1)=(n-1)n/2$	n^2
a[j]=a[j+1];	1	-	-	1	$(n-1)n/2$	n^2
a[j+1]=temp; }	1	-	-	1		
} } }	1			1		
Time Complexity			O(n)			O(n ²)

Time Complexity of Bubble Sort

Best Case : O(n)
Worst Case : O(n²)
Average Case : O(n²)

1.14. Optimized Version of Bubble Sort

```

void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    bool swapped;
    for (i = 0; i < n - 1; i++)
    {
        swapped = false;
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
                swapped = true;
            }
        }
        if (swapped == false)    // If no two elements were swapped by inner loop, then break
            break;
    }
}

```

1.15. Solving recurrence have three famous methods

- Substitution Method
- Recursive Tree Method
- Master Theorem

Substitution Method:

Substitution Method is very famous method for solving any recurrences. There are two types of substitution methods-

- ✓ Forward Substitution
- ✓ Backward Substitution

Forward Substitution:

It is called Forward Substitution because here we substitute recurrence of any term into next terms. It uses following steps to find Time using recurrences-

- ❖ Start with initial term
- ❖ Generate first few terms
- ❖ Identify any pattern that can be expressed by a close end formula
- ❖ Check its validity

Example 1

$$T(n) = 2T(n-1) + 1$$

$$T(1) = 1$$

Solution

$$\begin{aligned} T(1) &= 1 \\ T(2) &= 2T(2-1) + 1 \\ &= 2T(1) + 1 \\ T(3) &= 2T(3-1) + 1 \\ &= 2T(2) + 1 \\ &= 2*3 + 1 = 7 \\ T(4) &= 2T(4-1) + 1 \\ &= 2T(3) + 1 \\ &= 2*7 + 1 = 15 \end{aligned}$$

1,3,7,15

The Pattern is $2^n - 1$

$$T(n) = 2^n - 1 \in O(2^n)$$

Example 2

Ex-1 $T(n) = 2T(n-1) + 1$ for $n > 1$
 $T(1) = 1$

Solution:

$$T(1) = 1.$$

$n=2$ $T(2) = 2T(2-1) + 1$
 $= 2T(1) + 1$
 $= 2(1) + 1$

$$T(2) = 3$$

$n=3$ $T(3) = 2T(3-1) + 1$
 $= 2T(2) + 1$
 $= 2(3) + 1$

$$T(3) = 7$$

$n=4$

$$T(4) = 2T(4-1) + 1$$

$$= 2T(3) + 1$$

$$= 2(7) + 1$$

$$T(4) = 15$$

The pattern is

$$1, 3, 7, 15, \dots$$

Hence we must identify the above pattern as, $2^n - 1$

$$T(n) = 2^n - 1 \in \Theta(2^n).$$

Example 3:

$$T(n) = T(n-1) + n \quad n \geq 1.$$

$$T(0) = 0.$$

Solution:

$$n=0 \quad T(0) = 0$$

$$\begin{aligned} n=1 \quad T(1) &= T(1-1) + 1 \\ &= T(0) + 1 \\ &= 0 + 1 \end{aligned}$$

$$T(1) = 1$$

$$\begin{aligned} n=2 \quad T(2) &= T(2-1) + 2 \\ &= T(1) + 2 \\ &= 1 + 2 \\ &= 3 \end{aligned}$$

$$T(2) = 3$$

$$\begin{aligned} n=3 \quad T(3) &= T(3-1) + 3 \\ &= T(2) + 3 \\ &= 3 + 3 \end{aligned}$$

$$T(3) = 6$$

$$\begin{aligned} n=4 \quad T(4) &= T(4-1) + 4 \\ &= T(3) + 4 \\ &= 6 + 4 \end{aligned}$$

$$T(4) = 10$$

$$1, 3, 6, 10, \dots$$

Thus the pattern is,

$$T(n) = \frac{n(n+1)}{2}$$

$$= \frac{n^2}{2} + \frac{n}{2}$$

$$T(n) \in O(n^2)$$

Backward Substitution

Steps:

1. Start from the recurrence relation : $T(n) = T(n-1) + 1$
2. Express $T(n)$ in terms of $T(n-1), T(n-2), T(n-3), \dots$ until a common pattern is identified among all
3. Based on the identified pattern : Frame the General Equation
4. Now find a value for 'k' with respect to n, so that $T(n-k)$ equates to initial condition
5. Frame the solution by leading to a closed end formula or direct

Example 1:

$$T(n) = T(n-1) + n.$$

$$T(0) = 0$$

$$T(n) = T(n-1) + n \rightarrow (1)$$

Find $T(n-1)$

$$T(n-1) = T(n-1-1) + (n-1)$$

$$T(n-1) = T(n-2) + (n-1) \rightarrow (2)$$

Subs (2) in (1).

$$T(n) = T(n-2) + (n-1) + n \rightarrow (3)$$

Find $T(n-2)$:

$$T(n-2) = T(n-2-1) + (n-2)$$

$$T(n-2) = T(n-3) + (n-2) \rightarrow (4)$$

Subs (4) in (3).

$$T(n) = T(n-3) + (n-2) + (n-1) + n \rightarrow (5)$$

Find $T(n-3)$:

$$T(n-3) = T(n-3-1) + (n-3)$$

$$T(n-3) = T(n-4) + (n-3) \rightarrow (6)$$

Subs (6) in (5).

$$T(n) = T(n-4) + (n-3) + (n-2) + (n-1) + n \rightarrow (7)$$

Expressing in terms of i : Egn (7)

$$T(n) = T(n-i) + (n-i+1) + (n-i+2) + (n-i+3) + \dots + n.$$

Substitute i ^{first term} in such a way that, first terms equals to Initial condition.

$$T(n-i) = T(0).$$

$$\boxed{\begin{matrix} n-i=0 \\ n=i \end{matrix}}$$

$$\begin{aligned}
 T(n) &= T(0) + (i-i+1) + (i-i+2) + (i-i+3) \\
 &\quad + \dots + n \\
 &= T(0) + 1 + 2 + 3 + \dots + n \\
 T(n) &= 0 + 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} = \frac{n^2+n}{2} \in \boxed{\Theta(n^2)}
 \end{aligned}$$

Example 2

$$T(n) = T(n/3) + C$$

$$T(1) = 1$$

Solution: $T(n) = T(n/3) + C \rightarrow \textcircled{1}$

Find $T(n/3)$:

$$T(n/3) = T(n/3/3) + C$$

$$T(n/3) = T(n/9) + C \rightarrow \textcircled{2}$$

Subs. $\textcircled{2}$ in $\textcircled{1}$

$$T(n) = T(n/9) + C + C$$

$$T(n) = T(n/9) + 2C \rightarrow \textcircled{3}$$

Find $T(n/9)$:

$$T(n/9) = T(n/9/3) + C$$

$$T(n/9) = T(n/27) + C \rightarrow \textcircled{4}$$

Subs. $\textcircled{4}$ in $\textcircled{3}$

$$T(n) = T(n/27) + C + 2C$$

$$T(n) = T(n/27) + 3C \rightarrow \textcircled{5}$$

Find $T(n/27)$:

$$T(n/27) = T(n/27/3) + C$$

$$T(n/27) = T(n/81) + C \rightarrow \textcircled{6}$$

Subs. (6) in (5) .

$$T(n) = T(n/81) + c + 3c .$$

$$T(n) = T(n/81) + 4c \rightarrow (7)$$

Expressing in terms of i : Egn (7):

$$T(n) = T(n/3^i) + ic$$

Must equate first term to Initial

$$T(n/3^i) = 1$$

$$n = 3^i$$

$$\log 3^i = \log n .$$

$$i \log 3 = \log_3 n$$

$$i = \log n .$$

$$T(n) = T(n/n) + c \log n$$

$$T(n) = c \log n + 1 .$$

Example 3:

$$T(n) = T(n-1) + 1$$

$$T(0) = 0$$

Solution : Let,

$$T(n) = T(n-1) + 1$$

By backward substitution,

$$T(n-1) = T(n-2) + 1$$

$$T(n) = T(n-1) + 1$$

$$= (T(n-2) + 1) + 1$$

$$T(n) = T(n-2) + 2$$

$$\begin{aligned}
 \text{Again } T(n-2) &= T(n-2-1) + 1 \\
 &= T(n-3) + 1 \\
 \therefore T(n) &= T(n-2) + 2 \text{ becomes} \\
 &= (T(n-3) + 1) + 2 \\
 T(n) &= T(n-3) + 3 \\
 &\vdots \\
 T(n) &= T(n-k) + k \\
 \text{If } k &= n \text{ then equation (1) becomes} \\
 T(n) &= T(0) + n \\
 &= 0 + n \\
 T(n) &= n
 \end{aligned}$$

Example 4:

$$T(n) = 2T(n/2) + n$$

Solution : With $T(1) = 1$ as initial condition

$$T(n) = 2 \left(2T\left(\frac{n}{4}\right) + \frac{n}{2} \right) + n$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 2n$$

$$T(n) = 4 \left(2T\left(\frac{n}{8}\right) + \frac{n}{4} \right) + 2n$$

$$= 8T\left(\frac{n}{8}\right) + 3n$$

$$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + 3n$$

$\vdots$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$$

$$T(n) = n \cdot T\left(\frac{n}{n}\right) + \log n \cdot n$$

$$= n \cdot T(1) + n \cdot \log n$$

$$T(n) = n + n \log(n)$$

i.e. $T(n) \approx n \log n$

Hence in terms of big oh notation -

$$T(n) = O(n \log n)$$

If we assume $2^k = n$

$$\therefore T(1) = 1$$

Example 5:

$$T(n) = T(n-1) + n^4$$

$$T(n) = T(n-1) + n^4$$

By backward substitution method,

$$T(n) = [T(n-2) + (n-1)^4] + n^4$$

$$= T(n-2) + (n-1)^4 + n^4$$

$$= [T(n-3) + (n-2)^4] + (n-1)^4 + n^4$$

$$= T(n-3) + (n-2)^4 + (n-1)^4 + n^4$$

$$= [T(n-4) + (n-3)^4] + (n-2)^4 + (n-1)^4 + n^4$$

$$= T(n-4) + (n-3)^4 + (n-2)^4 + (n-1)^4 + n^4$$

$$\vdots$$

$$= T(n-k) + (n-k+1)^4 + (n-k+2)^4 + (n-k+3)^4 + \dots + n^4$$

If $k = n$ then

$$= T(n-n) + (n-n+1)^4 + (n-n+2)^4 + (n-n+3)^4 + \dots + n^4$$

$$= T(0) + 1^4 + 2^4 + 3^4 + \dots + n^4$$

$$= T(0) + \sum_{i=1}^n i^4$$

$$= T(0) + n(n^4)$$

$$= 0 + n^5$$

$$\therefore T(n) = \Theta(n^5)$$

$$\therefore T(n-1) = T(n-2) + (n-1)^4$$

$$\therefore \sum_{i=1}^n i^4 = n^4 \cdot \sum_{i=1}^n 1 = n^4 \cdot n$$

$\therefore$ Assume $T(0) = 0$ as initial condition

Recursion tree Method

What is Recursion Tree?

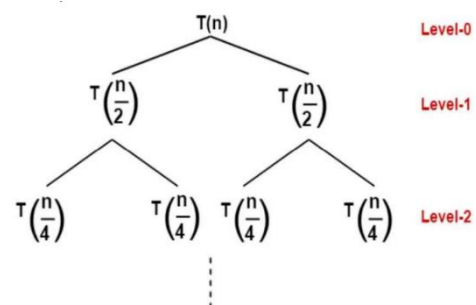
A recursion tree is a visual representation of the recursive calls made by a function.

- Used to devise a good guess
- Tree-like structure that shows the sequence of calls and the relationships between them.
- Root node: The initial function call
- Child nodes: Recursive calls made by the function
- Leaf nodes: Base cases that terminate the recursion
- Edges: Represent the recursive calls between nodes.
- Recursion trees help analyze and understand recursive algorithms

✓Time complexity: By counting the number of nodes

✓Space complexity: By looking at the maximum depth

- Termination: By checking if the tree has a finite number of nodes.



Recursion Tree Time Complexity Calculation

- Recursion Tree model the costs of a recursive execution of an algorithm
- A tree node represents the cost of a sub-problem
- Recursion Tree consists of 2 parts:

- (i) Cost of non-recursive part
- (ii) Cost of recursive call on smaller input size
- Total Cost = Cost of individual node at depth “i” + Sum up the cost of all nodes at depth “i” + Sum up all per-level costs of Recursion Tree
- (i.e.) Total cost = Cost of Leaf nodes (Lc) + Cost of Internal Nodes (Ic)

$$x^{\log_y n} \Rightarrow n^{\log_y x}$$

$$x^0 + x^1 + x^2 + \dots + x^n = \frac{x^{n+1} - 1}{x - 1} \quad \text{for } x \neq 1$$

$$x^0 + x^1 + x^2 + \dots = \frac{1}{1-x} \quad \text{for } |x| < 1$$

Example 1:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T\left(\frac{n}{2}\right) + n & n > 1 \end{cases}$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2}$$

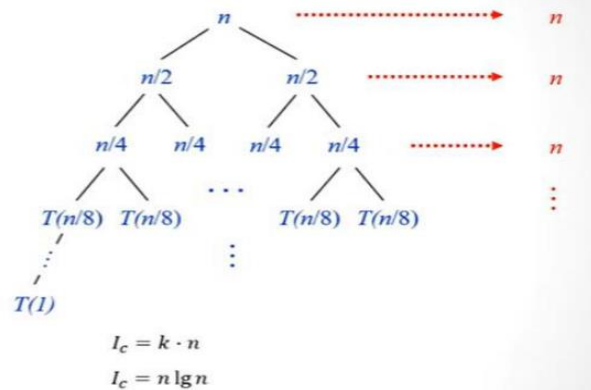
$$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}$$

$$T\left(\frac{n}{2^{k-1}}\right) = 2T\left(\frac{n}{2^k}\right) + \frac{n}{2^{k-1}}$$

$$T\left(\frac{n}{2^k}\right) = T(1)$$

$$n = 2^k \Rightarrow k = \lg n$$

$$L_c = 2^k \Rightarrow 2^{\lg n} \Rightarrow n^{\lg 2} \Rightarrow n$$



$$\text{Total Cost} = L_c + I_c \Rightarrow n + n \lg n$$

$$\text{Hence: } T(n) \in O(n \lg n)$$

Example 2:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T\left(\frac{n}{2}\right) + n^2 & n > 1 \end{cases}$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

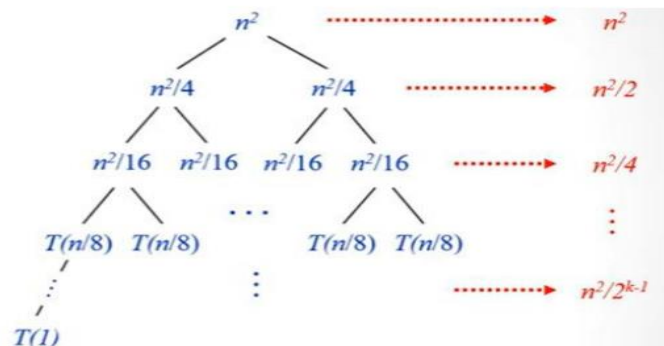
$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n^2}{2^2}$$

$$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n}{2^3}\right) + \frac{n^2}{4^2}$$

$$T\left(\frac{n}{2^k}\right) = T(1)$$

$$n = 2^k \Rightarrow k = \lg n$$

$$L_c = 2^k \Rightarrow 2^{\lg n} \Rightarrow n^{\lg 2} \Rightarrow n$$



$$\text{Total Cost} = L_c + I_c \Rightarrow n + 2n^2$$

$$\text{Hence: } T(n) \in O(n^2)$$

Example 3:

$$T(n) = \begin{cases} 1 & n = 1 \\ 3T\left(\frac{n}{4}\right) + n^2 & n > 1 \end{cases}$$

$$T(n) = 3T\left(\frac{n}{4}\right) + n^2$$

$$T\left(\frac{n}{4}\right) = 3T\left(\frac{n}{4^2}\right) + \frac{n^2}{4^2}$$

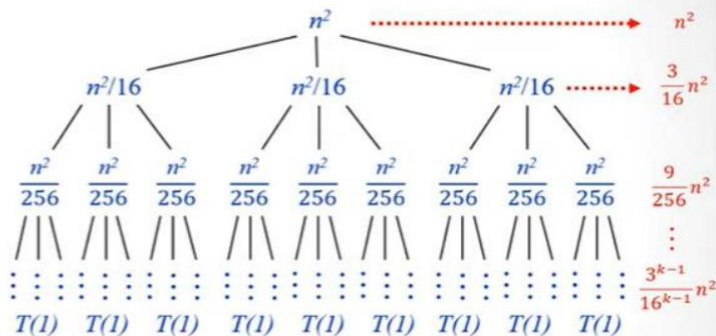
$$T\left(\frac{n}{4^2}\right) = 3T\left(\frac{n}{4^3}\right) + \frac{n^2}{16^2}$$

$$T\left(\frac{n}{4^k}\right) = T(1)$$

$$n = 4^k$$

$$k = \log_4 n$$

$$L_c = 3^k \Rightarrow 3^{\log_4 n} \Rightarrow n^{\log_4 3}$$



$$I_c = n^2 \cdot \left[\left(\frac{3}{16}\right)^0 + \left(\frac{3}{16}\right)^1 + \left(\frac{3}{16}\right)^2 + \dots + \left(\frac{3}{16}\right)^{k-1} \right]$$

$$I_c = n^2 \cdot \left[\frac{1}{1 - 3/16} \right] \Rightarrow \frac{16}{13} n^2$$

$$\text{Total Cost} = L_c + I_c \Rightarrow n^{\log_4 3} + \frac{16}{13} n^2$$

Hence: $T(n) \in O(n^2)$

Example 4:

$$T(n) = \begin{cases} 1 & n = 1 \\ T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2 & n > 1 \end{cases}$$

$$T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$$

$$T\left(\frac{n}{4}\right) = T\left(\frac{n}{16}\right) + T\left(\frac{n}{8}\right) + \frac{n^2}{16}$$

$$T\left(\frac{n}{2}\right) = T\left(\frac{n}{8}\right) + T\left(\frac{n}{4}\right) + \frac{n^2}{4}$$

$$T\left(\frac{n}{16}\right) = T\left(\frac{n}{64}\right) + T\left(\frac{n}{32}\right) + \frac{n^2}{256}$$

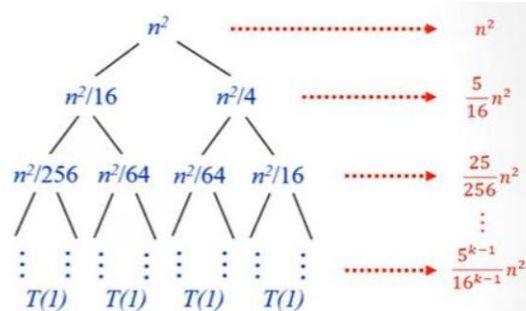
$$T\left(\frac{n}{8}\right) = T\left(\frac{n}{32}\right) + T\left(\frac{n}{16}\right) + \frac{n^2}{64}$$

$$T\left(\frac{n}{4}\right) = T\left(\frac{n}{16}\right) + T\left(\frac{n}{8}\right) + \frac{n^2}{16}$$

$$T\left(\frac{n}{2^k}\right) = T(1)$$

$$n = 2^k \Rightarrow k = \lg n$$

$$L_c = 2^k \Rightarrow 2^{\lg n} \Rightarrow n$$



$$I_c = n^2 \cdot \left[\left(\frac{5}{16}\right)^0 + \left(\frac{5}{16}\right)^1 + \left(\frac{5}{16}\right)^2 + \dots + \left(\frac{5}{16}\right)^{k-1} \right]$$

$$I_c = n^2 \cdot \left[\frac{1}{1 - 5/16} \right] \Rightarrow \frac{16}{11} n^2$$

$$\text{Total Cost} = L_c + I_c \Rightarrow n + \frac{16}{11} n^2$$

Hence: $T(n) \in O(n^2)$

1.16. Masters Theorem for Decreasing Functions

Consider a relation of type –

$T(n) = aT(n-b) + f(n)$, where, $a \geq 1$ and $b > 1$, $f(n)$ is asymptotically positive

Here,

n – size of the problem

a – number of sub-problems in the recursion

$n-b$ – size of the sub problems based on the assumption that all sub-problems are of the same size.

$f(n)$ – represents the cost of work done outside the recursion $\rightarrow (n^k)$, where $k \geq 0$.

$$T(n) = aT(n-b) + f(n),$$

If the recurrence relation is in the above given form, then there are three cases in the master theorem to determine the asymptotic notations –

- if $a = 1$, $T(n) = O(n^{k+1})$
- if $a > 1$, $T(n) = O(a^{n/b} * n^k)$
- if $a < 1$, $T(n) = O(n^k)$

Example 1: Consider a recurrence relation given as $T(n) = T(n-1) + n^2$

In this problem, $a = 1$, $b = 1$ and $f(n) = O(n^k) = n^2$, giving us $k = 2$.

Since $a = 1$, case 1 must be applied for this equation.

To calculate, $T(n) = O(n^{k+1})$

$$= n^{2+1}$$

$$= n^3$$

Therefore, $T(n) = O(n^3)$ is the tight bound for this equation.

Example 2: Consider a recurrence relation given as $T(n) = 2T(n-1) + n$

In this problem, $a = 2$, $b = 1$ and $f(n) = O(n^k) = n$, giving us $k = 1$.

Since $a > 1$, case 2 must be applied for this equation.

To calculate, $T(n) = O(a^{n/b} * n^k)$

$$= O(2^{n/1} * n^1)$$

$$= O(n2^n)$$

Therefore, $T(n) = O(n2^n)$ is the tight bound for this equation.

Example 3: Consider a recurrence relation given as $T(n) = n^4$

In this problem, $a = 0$ and $f(n) = O(n^k) = n^4$, giving us $k = 4$

Since $a < 1$, case 3 must be applied for this equation.

To calculate, $T(n) = O(n^k)$

$$= O(n^4)$$

$$= O(n^4)$$

Therefore, $T(n) = O(n^4)$ is the tight bound for this equation.

1.17. Merge Sort - Iterative

In Iterative merge sort, we implement merge sort in a bottom-up manner. This is how it works:

- We start by sorting all sub-arrays of length 1
- Then, we sort all sub-arrays of length 2 by merging length-1 sub-arrays
- Then, we sort all sub-arrays of length 4 by merging length-2 sub-arrays
- We repeat the above step for sub-arrays of lengths 8, 16, 32, and so on until the whole array is sorted

How Does Iterative Merge Sort Work?

Let's assume that the array $Arr[] = \{3, 2, 1, 9, 5, 4, 10, 11\}$ of size $N = 8$ is to be sorted.

Arrays of length 1 are trivially sorted.

Step 1: First, we take $sub_size = 1$ and merge all pairs of sub-arrays of size 1.

Step 2: Then, we multiply sub_size by 2, and sub_size becomes 2. Now, we merge all pairs of sub-arrays of size 2.

Step 3: Again, we multiply sub_size by 2, and sub_size becomes 4, and we merge all pairs of sub-arrays of size 4.

Now, we stop, as sub_size is $\geq N$ and the array is sorted.

```
void merge(int Arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++)
        L[i] = Arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = Arr[m + 1 + j];
    i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
```

```

    if (L[i] <= R[j])
    {
        Arr[k] = L[i]; i++;
    }
    else {
        Arr[k] = R[j]; j++;
    }
    k++;
}
while (i < n1) {
    Arr[k] = L[i];
    i++;
    k++;
}
while (j < n2) {
    Arr[k] = R[j];
    j++;
    k++;
}
}
void merge_sort(int Arr[], int N){
    for(int sub_size=1;sub_size<N;sub_size*=2)
    {
        for(int L=0; L<N; L+=(2*sub_size))
        {
            int Mid=min(L+sub_size-1,N-1);
            int R=min(L+2*sub_size-1,N-1);
            merge(Arr, L,Mid,R);
        }
    }
}

```

Time Complexity Analysis

1. Number of Iterations

- The array size starts with **subarrays of size 1**.
- Each iteration **doubles** the size of the subarrays.
- The number of iterations is determined by how many times we can **double the size** before reaching n.
- Since curr_size starts at 1 and doubles ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots$), the number of iterations is: **$O(\log_{2} n)$**

2. Merging Process in Each Iteration

- Every iteration merges all elements once.
- Merging two sorted subarrays of total size n takes **$O(n)$** time.
- Since merging happens in every iteration, the total merging time is: $O(n)$ per iteration

Overall Time Complexity

Since the process runs **$O(\log n)$** times and each iteration takes **$O(n)$** time: **$O(n \log n)$**

Advantages of Iterative Merge Sort

- Worst-case, best-case, and average-case time complexity of merge sort are $O(N \log N)$, making it very efficient.
- Iterative merge sort is slightly faster than recursive merge sort.

Disadvantages of Iterative Merge Sort

- Space complexity of iterative merge sort is $O(N)$